

Skyscraper Stack

Development session with Claude

A Pygame tower-stacking game – bug fix, Perfect Placement combos, falling debris, and an altitude-driven sky. Full session transcript, October 8, 2026.

ORIGINAL CONVERSATION
<https://claude.ai/share/0912855e-c745-4a21-be8d-8730f22c7e7a>

PROJECT REPOSITORY
https://github.com/Edwin-Himax/54_skyscrapper_stack

Session overview

Part	Request	Outcome
1	Find why the overlap check in <code>drop_block()</code> is inverted	Root cause: <code>overlap <= 0</code> should be <code>overlap > 0</code> ; corrected method and explanation of the 1D intersection math
2	Perfect Placement: snap tolerance, bonus popups, combo width restore	Tested implementation in <code>game_engine.py</code> with <code>FloatingText</code> popups
3	Falling off-cut debris animation	New <code>game/debris.py</code> (<code>Debris</code> class) hooked into update/render, camera scroll and misses
4	Dynamic sky colors by tower height	Keyframed lerp gradient, cached surface, twinkling stars; measured at 0.07–1.1 ms/frame
5	Export this session as a PDF with a link	This document

Files touched: `game/game_engine.py` (all parts) and the new `game/debris.py` (Part 3).

Part 1 - Fixing the inverted overlap logic

YOU

https://github.com/Edwin-Himax/54_skyscraper_stack/tree/main/game

I am working on a Pygame tower stacking game called "Skyscraper Stack".

Currently, when I land a block on top of the tower, it triggers an immediate "TOWER COLLAPSED!" game over. Conversely, dropping a block in empty air lets the game continue.

Please analyze `game/game_engine.py` (specifically around `drop_block()`) and:

1. Identify why the overlap condition logic is inverted.
2. Explain the 1D bounding box intersection math (`left = max(...)`, `right = min(...)`, `overlap = right - left`).
3. Provide the corrected code for `drop_block()` so valid overlaps stack properly and complete misses trigger game over.
4. Keep the code clean and well-commented.

CLAUDE

I cloned your repo and read `game/game_engine.py`. The bug is on line 61.

1. Why the logic is inverted

```
is_successful_drop = overlap <= 0  # ← this is the bug
```

`overlap <= 0` is true when the blocks don't overlap at all (a gap or just touching), yet the code treats that as a successful drop. So:

- **Landing on the tower** gives `overlap > 0`, so `is_successful_drop` is `False` and you hit the `else` branch (`game_over = True`).
- **Dropping in empty air** gives `overlap <= 0`, so `is_successful_drop` is `True` and the game builds a block from a non-existent overlap and continues.

The condition should be `overlap > 0`.

2. The 1D bounding-box intersection math

Since blocks only move horizontally, the check reduces to two intervals on the x-axis: the active block `[act.x, act.x + act.width]` and the top block `[top.x, top.x + top.width]`.

```
left   = max(act.x, top_block.x)           # the later start
right  = min(act.x + act.width, top_block.x + top_block.width)  # the earlier end
overlap = right - left
```

Where two intervals overlap, the shared region starts at whichever left edge is further right (`max`) and ends at whichever right edge is further left (`min`). The length of that region is `right - left`.

For example, with the top block at `[100, 280]` and the active block at `[150, 330]`:

- `left = max(150, 100) = 150`
- `right = min(330, 280) = 280`
- `overlap = 130`, so the new block is 130 px wide, trimmed to the shared region.

The sign of `overlap` tells you the case:

overlap	Meaning
<code>> 0</code>	The blocks share some horizontal span, so it's a valid stack
<code>== 0</code>	The edges just touch, so it's a miss (no support)
<code>< 0</code>	There's a gap, and the magnitude is the gap size, so it's a miss

3. Corrected `drop_block()`

```
def drop_block(self):
```

```

"""Attempt to land the active block on top of the stack.

Uses 1D interval intersection on the x-axis:
- overlap > 0 -> the block is at least partly supported: trim it to
    the shared region, add it to the tower, and continue.
- overlap <= 0 -> a complete miss: the tower collapses (game over).
"""
if self.game_over:
    return

top_block = self.stack[-1]
act = self.active_block

# Intersection of [act.x, act.x + act.width] and [top_block.x, top_block.x + top_block.width]
left = max(act.x, top_block.x) # later left edge
right = min(act.x + act.width, top_block.x + top_block.width) # earlier right edge
overlap = right - left # <= 0 means no overlap

has_support = overlap > 0 # FIX: was `overlap <= 0`, which was inverted

if has_support:
    # Trim the new block to exactly the overlapping span
    new_block = Block(left, act.y, overlap, self.block_height, act.color, speed=0)
    self.stack.append(new_block)
    self.score += 1

    # Camera scroll: once the tower gets high, shift everything down
    if new_block.y < 180:
        shift_amount = self.block_height + 4
        for b in self.stack:
            b.y += shift_amount

    self.spawn_active_block()
else:
    # Complete miss: nothing to rest on
    self.game_over = True

```

4. Other things I noticed

- **The `max(10.0, overlap)` floor.** I removed it. With the old inverted check it was masking the geometry, but now it causes a visual bug: if the overlap is, say, 3 px, the block is widened to 10 px starting at `left`, so it hangs off the edge of the tower. If you want a minimum playable width, the cleaner way is to treat a tiny overlap as a miss, e.g. `has_support = overlap >= 10`.
- **Restart key mismatch.** The Game Over screen says "Press [Space] or [R] to Play Again", but `handle_event` only restarts on `R` or a left click. Space won't work. Add `or event.key == pygame.K_SPACE` to the `KEYDOWN` check on line 80 if you want the text to be accurate.

Part 2 - Perfect Placement mechanics

YOU

Add a "Perfect Placement" mechanics feature to my Pygame skyscraper stacking game in `game/game_engine.py`.

Requirements:

1. **Alignment Tolerance:** If the offset between the active block and top stack block is within a small threshold (e.g., ≤ 4 pixels), snap the active block perfectly flush with the stack below it (0 width loss).
2. **Bonus Points & UI Feedback:** Show an animated floating visual indicator (e.g., "+50 PERFECT!") near the dropped block that rises and fades out.
3. **Width Restoration Combo System:** Track consecutive perfect drops. If the player gets 3 consecutive perfect drops in a row, slightly expand/restore the block width (up to the base maximum width) as a reward.

Please provide the updated `drop_block()`, `update()`, and `render()` logic for `game/game_engine.py` with modular helper methods.

CLAUDE

I added Perfect Placement to your `game_engine.py` and tested it headlessly against your real code. The full updated file was attached, and it drops in over `game/game_engine.py`.

The tests confirmed:

- An offset of 3 px snaps flush with no width loss.
- An offset of 5 px trims as normal and resets the streak.
- The third perfect in a row restores 12 px of width, centered on the block.
- Width never exceeds `base_width`.
- Popups rise, fade and are removed.
- A miss still ends the game.
- `reset()` clears the new state.

My first pass also had a bug, which the tests caught: `_render_hud` was inserted into the wrong class. It was fixed before delivery.

Tuning constants and popup class

```
class FloatingText:
    """A short-lived text popup that rises and fades out (frame-based)."""

    def __init__(self, text, font, color, center_x, y, lifetime=60, rise_speed=1.2):
        self.surface = font.render(text, True, color) # pre-rendered once
        self.x = center_x - self.surface.get_width() / 2
        self.y = float(y)
        self.age = 0
        self.lifetime = lifetime
        self.rise_speed = rise_speed

    @property
    def alive(self):
        return self.age < self.lifetime

    def update(self):
        self.age += 1
        self.y -= self.rise_speed

    def render(self, screen):
        alpha = max(0, int(255 * (1 - self.age / self.lifetime))) # linear fade
        self.surface.set_alpha(alpha)
        screen.blit(self.surface, (self.x, self.y))

class GameEngine:
    PERFECT_TOLERANCE = 4 # max px offset that still counts as "perfect"
    PERFECT_BONUS = 50
    COMBO_THRESHOLD = 3 # perfects in a row needed for the width reward
```

```
WIDTH_RESTORE = 12          # px restored per reward
```

drop_block() and helpers

```
def _compute_landing(self, act, top):
    """Returns (x, width, is_perfect), or None on a complete miss."""
    # Active and top blocks share a width, so the left-edge offset is the misalignment
    if abs(act.x - top.x) <= self.PERFECT_TOLERANCE:
        return top.x, top.width, True          # snap flush: zero width loss

    left = max(act.x, top.x)
    right = min(act.x + act.width, top.x + top.width)
    overlap = right - left
    if overlap <= 0:
        return None                            # miss
    return left, overlap, False

def _restore_width(self, x, width):
    """Combo reward: widen (centered), capped at base width. Returns (x, width, restored)."""
    new_width = min(float(self.base_width), width + self.WIDTH_RESTORE)
    if new_width <= width:
        return x, width, False                # already at max
    new_x = x - (new_width - width) / 2
    new_x = max(20, min(new_x, self.width - 20 - new_width))  # stay on screen
    return new_x, new_width, True

def _scroll_camera_if_needed(self, new_block):
    if new_block.y < 180:
        shift = self.block_height + 4
        for b in self.stack:
            b.y += shift
        for t in self.floating_texts:          # keep popups attached to the scene
            t.y += shift

def _spawn_perfect_feedback(self, block, restored):
    cx = block.x + block.width / 2
    self.floating_texts.append(FloatingText(
        f"+{self.PERFECT_BONUS} PERFECT!", self.font_hud, (255, 230, 90), cx, block.y - 26))
    if restored:
        self.floating_texts.append(FloatingText(
            "WIDTH RESTORED!", self.font_hud, (90, 230, 140), cx, block.y - 50, lifetime=75))

def drop_block(self):
    if self.game_over:
        return

    top_block, act = self.stack[-1], self.active_block
    landing = self._compute_landing(act, top_block)
    if landing is None:
        self.combo = 0
        self.game_over = True
        return

    x, width, is_perfect = landing
    restored = False
    if is_perfect:
        self.combo += 1
        self.bonus += self.PERFECT_BONUS
        if self.combo % self.COMBO_THRESHOLD == 0:  # every 3rd perfect in a row
            x, width, restored = self._restore_width(x, width)
    else:
        self.combo = 0                            # streak broken

    new_block = Block(x, act.y, width, self.block_height, act.color, speed=0)
    self.stack.append(new_block)
    self.score += 1

    self._scroll_camera_if_needed(new_block)      # before feedback, so it isn't shifted
    if is_perfect:
        self._spawn_perfect_feedback(new_block, restored)
    self.spawn_active_block()
```

update() and render()

```
def _update_floating_texts(self):
    for t in self.floating_texts:
        t.update()
    self.floating_texts = [t for t in self.floating_texts if t.alive]

def update(self):
    self._update_floating_texts()                # keep animating even after game over
    if not self.game_over:
```

```

self.active_block.update(self.width)

def _render_hud(self, screen):
    score_surf = self.font_hud.render(
        f"Height: {self.score} Bonus: {self.bonus}", True, (255, 220, 80))
    screen.blit(score_surf, (self.width // 2 - score_surf.get_width() // 2, 54))
    if self.combo > 0:
        combo_surf = self.font_hud.render(
            f"Perfect streak x{self.combo}", True, (90, 230, 140))
        screen.blit(combo_surf, (self.width // 2 - combo_surf.get_width() // 2, 80))

# In render(): replace the old score blit with self._render_hud(screen), and draw
# popups after the blocks but before the game-over overlay:
#     for t in self.floating_texts:
#         t.render(screen)

```

`reset()` also needs `self.bonus = 0`, `self.combo = 0` and `self.floating_texts = []`. The attached file already had them.

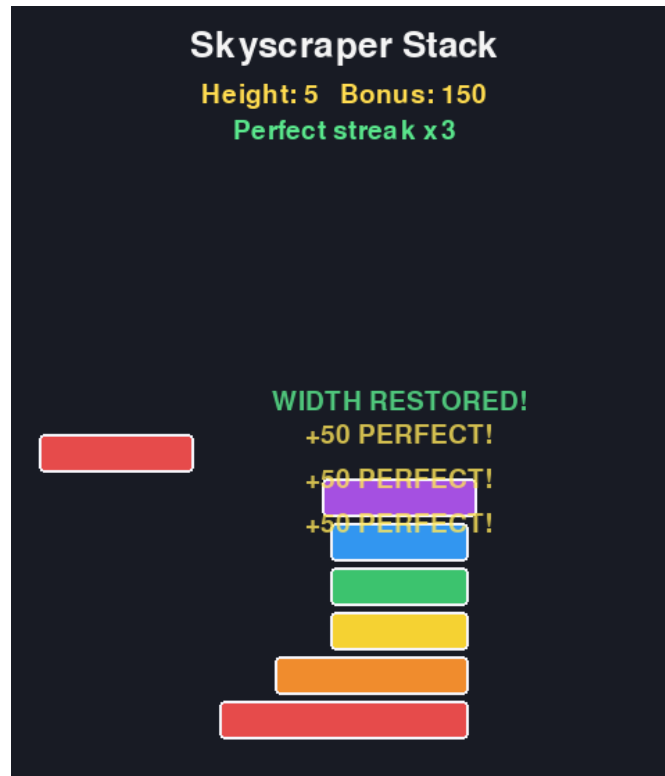


Figure 1 – Test render: "+50 PERFECT!" and "WIDTH RESTORED!" popups above the tower, with the bonus total and perfect-streak counter in the HUD.

Design decisions to check

- **Bonus is separate from height.** `score` still means tower height, so "Height" stays accurate. Bonus is shown alongside it. If you'd rather have a single total, use `self.score + self.bonus` in the HUD.
- **The reward repeats.** `combo % 3 == 0` rewards streaks of 3, 6, 9 and so on, rather than only once. Change it to `self.combo == self.COMBO_THRESHOLD` and reset the combo afterward for a one-time reward.
- **The popups are frame-based.** This matches your 60 FPS loop and has no timers. The camera scroll also shifts live popups, so they don't drift when the tower scrolls.
- **Tolerance can still be a miss near a screen edge.** The snap uses `top.x` directly, so a perfect block always lands exactly on the tower.

Part 3 - Falling off-cut debris

YOU

I want to implement a falling off-cut debris animation whenever a block is trimmed in my Pygame stacking game.

Requirements:

1. Create a `Debris` class in `game/block.py` or `game/debris.py` with: position (`x`, `y`), width, height, and color; gravity/velocity (`vy`, `vx`), angular rotation, and opacity/alpha.
2. **Trigger Logic:** When an active block overhangs and gets sliced, instantiate a `Debris` object for the sliced piece (either left or right slice depending on overhang direction).
3. **Physics & Rendering:** Debris should fall downward, rotate slightly, accelerate with gravity, fade out, and be removed once off-screen or faded.
4. **Clean Integration:** Show how to hook this seamlessly into `game_engine.py`'s update and render loop.

Please write clean, object-oriented code with comments.

CLAUDE

Off-cut debris is in and works. When a block is trimmed, the sliced piece tumbles away, fades out and is removed. The new `game/debris.py` and the updated `game/game_engine.py` were attached, and both keep your CRLF line endings.

I tested it headlessly: left and right slices, perfect drops, the physics, removal and a miss all behave correctly. In the mid-fall screenshot, the blue piece in the lower left is a freshly sliced off-cut that is still solid, and the gold piece in the lower right has already rotated and started to fade.

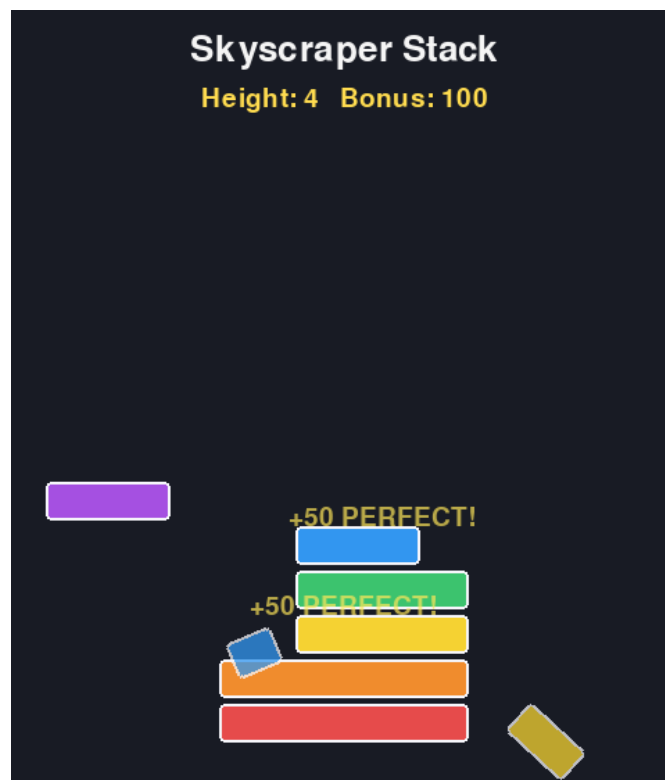


Figure 2 - Mid-fall frame: two off-cuts tumbling away from the tower, one already fading.

The `Debris` class

`game/debris.py` holds all the physics and rendering. Each frame, `update()` does this:

```
self.vy += self.gravity          # accelerate downward
self.x += self.vx                # drift outward
self.y += self.vy
self.angle += self.angular_velocity
self.alpha = max(0.0, self.alpha - self.fade_rate)
```

- **Rendering:** the piece is drawn once into a small surface with the same border style as `Block`. Each frame, `render()` rotates it about its center and blits it with the current alpha. Rotating about the center means the piece tumbles in place instead of orbiting a corner.
- **Removal:** `is_alive(screen_height)` returns `False` once the piece has fully faded or dropped below the screen.

Hooking it into `game_engine.py`

The engine changes are small, and each one reuses a pattern from the popups.

- **Import and state:** add `from game.debris import Debris` and `self.debris = []` in `reset()`.
- **Trigger on trim:** in `drop_block()`'s non-perfect branch, `self._spawn_debris(act, x, width)` compares the active block's span with the kept span. Whatever sticks out on the left or right becomes a `Debris` piece with the right width and position. A perfect snap spawns nothing.
- **Direction:** a left off-cut drifts left (`vx < 0`) and rotates counter-clockwise, so its outer edge dips like it's tipping over the edge. A right off-cut does the mirror image.
- **Update:** `update()` calls the new `_update_debris()`, which steps every piece and filters out dead ones. It runs even after game over, so pieces keep falling on the collapse screen.
- **Render:** `render()` draws debris after the blocks and before the popups and game-over overlay, so pieces fall in front of the tower.
- **Camera scroll:** `_scroll_camera_if_needed()` now shifts debris down along with the stack and popups, so pieces don't drift when the tower scrolls.

Things to know

- **Misses also spawn debris.** On a complete miss, the whole block tumbles away toward the side it missed on, instead of vanishing. That was one extra line, and it makes the game-over moment feel better. To remove it, delete the `_spawn_miss_debris(...)` call in `drop_block()`.
- **Tuning knobs.** All are constructor arguments on `Debris`, with defaults of `gravity=0.45` and `fade_rate=3.5`. At 60 FPS a piece fades fully in about 70 frames. In practice the tuning keeps most pieces visible until they near the bottom of the screen. The drift and spin ranges are in `_make_debris()` in `game_engine.py`.
- **Slivers are ignored.** Off-cuts of 0.5 px or less are skipped, since they would be invisible.

Part 4 - Dynamic sky color transitions

YOU

I would like to implement dynamic sky color transitions based on tower height in `game/game_engine.py`.

Requirements:

1. **Color Palettes & Progression:** transition smoothly through atmospheric stages as tower height increases:

- Altitude 0 - 10: Afternoon Day Blue (e.g., #4A90E2)
- Altitude 10 - 25: Golden Sunset (e.g., #E65C00 -> #762B80)
- Altitude 25 - 45: Twilight Dusk (e.g., #2C3E50 -> #0F2027)
- Altitude 45+: Deep Space Dark with subtle twinkling background stars.

2. **Smooth Interpolation:** use Linear Interpolation (lerp) between RGB color tuples based on `self.score`.

3. **Performance:** ensure the gradient generation or background fill runs efficiently inside `render(screen)` at 60 FPS.

Please provide the updated background rendering function for `GameEngine`.

CLAUDE

The sky now changes with tower height, and the updated `game_engine.py` was attached with everything from the earlier features still in place. I tested it headlessly and rendered six altitudes side by side. They show the afternoon blue turning to sunset, then dusk, then star-filled space.



Figure 3 - The sky at altitudes 0, 17, 25, 35, 45 and 52 (left to right).

The background logic is these methods, all in `GameEngine`:

```
def _sky_colors(self, altitude):
    """Lerp the (top, bottom) sky colors for an altitude using SKY_KEYFRAMES."""
    frames = self.SKY_KEYFRAMES
    if altitude <= frames[0][0]:
        return frames[0][1], frames[0][2]
    for (a0, top0, bot0), (a1, top1, bot1) in zip(frames, frames[1:]):
        if altitude <= a1:
            t = (altitude - a0) / (a1 - a0)
            return lerp_color(top0, top1, t), lerp_color(bot0, bot1, t)
    return frames[-1][1], frames[-1][2]

def _refresh_sky_surface(self):
    """Rebuild the cached gradient only when its colors actually change."""
    top, bottom = self._sky_colors(self.sky_altitude)
    if (top, bottom) == self._sky_key:
        return
    self._sky_key = (top, bottom)
    last = self.height - 1
    for y in range(self.height):
        self._sky_strip.set_at((0, y), lerp_color(top, bottom, y / last))
    pygame.transform.scale(self._sky_strip, (self.width, self.height), self._sky_surface)

def _render_background(self, screen):
    """Draw the altitude-based sky: cached gradient, then stars on top."""
    self._refresh_sky_surface()
    screen.blit(self._sky_surface, (0, 0))
    self._render_stars(screen)
```

`render()` now calls `_render_background(screen)` instead of the flat `screen.fill`. The file also has the supporting pieces: the keyframe table, `lerp/lerp_color/hex_to_rgb`, the star field, `_star_visibility`, `_render_stars` and `_update_sky_altitude`.

How the stages are mapped

Your stage list could be read in two ways, so here is what I chose. Each keyframe is a top-and-bottom color pair, and the sky lerps between consecutive keyframes:

Altitude	Sky
0-10	Day blue held flat (#4A90E2 top, a lighter blue at the horizon)
10 → 25	Blends to golden sunset: #E65C00 at the horizon, #762B80 at the top
25 → 45	Blends to twilight dusk: #2C3E50 at the horizon, #0F2027 at the top
45 → 50	Settles into deep space, and the stars fade in
50+	Deep space, stars fully visible

I read your A -> B arrows as horizon-to-zenith colors within the sky. As a result, each stage reaches its full look at its upper bound (full sunset at 25, full dusk at 45), and the sky is blending toward it throughout the stage. If you'd rather have a stage show its full palette from its lower bound, edit the altitudes in SKY_KEYFRAMES. Nothing else needs to change.

Smoothness and performance

- **Lerp on score:** the color blend is driven by sky_altitude, a smoothed copy of self.score. Each drop makes the sky glide over about a second instead of stepping. Set SKY_EASE = 1.0 to follow the score instantly.
- **Cached gradient:** the gradient is built as a 1-pixel column and stretched across the width with one scale call. It is rebuilt only when the RGB colors actually change. During the held stages, and between sub-pixel easing steps, a frame costs just one blit.
- **Stars:** there are 70 of them, and they twinkle with a sine wave of different speed and phase for each star. They fade in from altitude 45 to 50 and cost nothing below 45. The layout comes from a private seeded RNG, so the sky looks the same every run and doesn't disturb your game's random.

Measured per frame, against a 16.7 ms budget at 60 FPS:

Case	Time
Held stage, cached	0.07 ms
Space with stars	0.13 ms
Worst case, gradient rebuilt every frame	1.1 ms
Whole update and render with a 12-block tower	0.5 ms

reset() snaps the sky back to day blue, so a restart doesn't replay the transition.

Part 5 - Exporting this session

YOU

Extract this entire session as a pdf and also add this link to that pdf

<https://claude.ai/share/0912855e-c745-4a21-be8d-8730f22c7e7a>

CLAUDE

This document. It contains each request and response from the session, the code that was provided, and the screenshots from the tests. The conversation link you supplied is on the title page and below.

ORIGINAL CONVERSATION

<https://claude.ai/share/0912855e-c745-4a21-be8d-8730f22c7e7a>